AI-Powered Fire Prediction Spread and Containment System

Software Architecture Specifications

Panic! At The Kernel
Client: EPI-USE Africa

September 2026

Full Name	Student Number
Megan Lai	u23608821
Inge Keyser	u23544563
Janri du Toit	u23632730
Ryan Lynn	u23541912
Marco de Wit	u22811975

Contents

1 Introduction

This Software Architecture Specifications document is for the AI-Powered Fire Spread Prediction and Containment System, Fire Away, and specifies how the system of this application is structured to satisfy the Systems Requirements Specifications.

2 Architectural Requirements

2.1 Architectural Patterns

2.1.1 3-Layer Architecture

The Presentation Layer, Control Layer, and Service Layer make up the 3 Layers of the layered architecture. The Service Layer sends data through the message queue to the Control Layer that sends report data to the Presentation Layer.

2.1.2 Blackboard Architecture

The Spread Model Engine, Terrain Processor, and Weather Data Adapter all connect to the blackboard to read and write data from a shared repository to collaboratively predict the spread of fires.

2.1.3 Message Queue

The event-driven, asynchronous message queue sits between the Control Layer and Service Layer, and decouples the Incident Manager and Authentication Controller from each other.

2.1.4 Edge Node - Security Proxy Pattern

Parallel to Presentation layer. Validates traffic before it reaches the control layer. This minimises access channels and limit exposure, enhancing security.

2.1.5 Adapter Pattern

This wraps the external Weather Data. It blocks the rest of the system from seeing the specifics of the external API.

2.1.6 Circuit Breaker Pattern

State - Closed:
All weather API calls passes through as normal. The adapter will track consecutive failures. After (e.g. 3 consecutive fails in 60s, trip circuit)

State - Open:
All calls to external services are blocked and the adapter falls back onto cache. After (e.g. 30s move state to Recovery)

State - Recovery:
Admit a probe request to the external weather API, if this passes the circuit state returns to closed.

2.2 Architectural Diagram

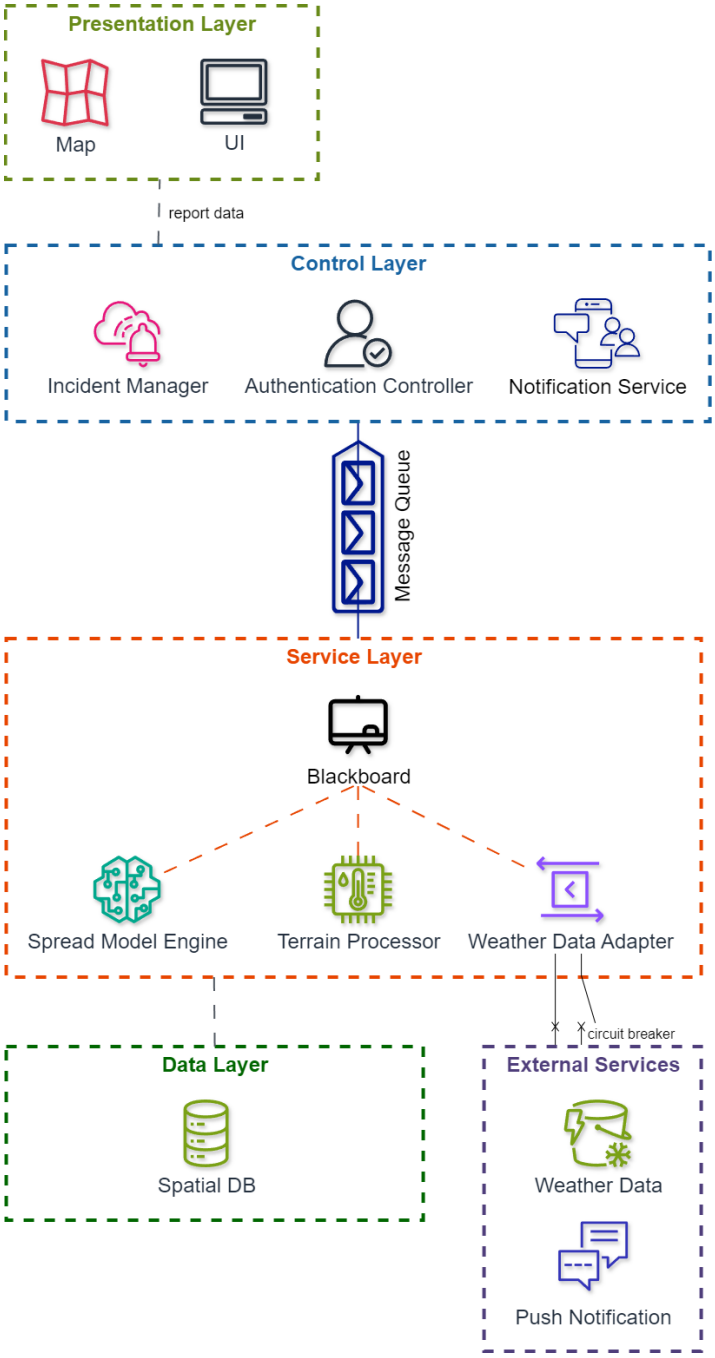


Figure 1: Architecture Diagram

2.3 Design Patterns

2.3.1 Singleton

- We use a singleton for the database engine and the session factory. The singleton pattern enforces that only a single instance of an object exists. Having only one instance of the database engine and session factory ensures that the application only has one source of truth and cannot respond with outdated or incorrect data across concurrent requests.

2.3.2 Pipeline

- We use the pipeline design pattern for the fire spread prediction engine. Each stage of the engines pipeline will be fed data from the previous step/stage which will transform the data and send it to the next step/stage as output. When a fire is reported and verified the live weather data is fetched for that fire and the data is passed to the XGBoost model which calculates an ignition probability for each tile, which is then passed to the DCA to make the final state change for a tile on the map and rasterize that tile.

2.3.3 Service Layer

- This pattern is used for the structure of our backend as it allows for the separation of concerns. We have a controller, which handles requests, and consits of our route files. Our database which stores everything and our service layer which is our services files which handle the actual logic behind the requests

2.3.4 Strategy

- We will use the strategy pattern for our risk scoring engine and our spread engine as it will allow us to have a common interface for the engines and allow us to have different concrete implementations of an engine. This will be beneficial to the project as it will allow us to add and swap engine implementations as needed for testing or if we decide to switch to another model for the engine.

2.3.5 Decorator

- The decorator pattern is used with the FastAPI backend router functions such as `@router.post()` and `@router.get()`. The route decorators wrap endpoint functions to add HTTP routing, request parsing, and response serialization without modifying the actual functions themselves.

2.3.6 Observer

- The observer pattern will be used for event-driven notifications across the system. The uses will be to notify users of a fire becoming verified, a nearby fire push notification for users to alert them of nearby fires, and the completion of a long spread prediction simulation. This pattern is the best choice as it allows many devices to be notified based on the state change of a single object which will allow multiple users to receive updates when the states mentioned above change.

2.4 Constraints

2.4.1 Resources From The Client

Data Sets recommended but not limited to the following:

- **Fire Occurrence & Historical Fires**

  - NASA Global Fire Atlas (2003–2016) – Fire dynamics, duration, spread patterns: https://data.nasa.gov/dataset/global-fireatlas-with-characteristics-of-individual-fires-2003-2016-69d65
  - NASA FIRMS Active Fire Data (MODIS & VIIRS) – Near real-time fire hotspots: https://firms.modaps.eosdis.nasa.gov/map/#d:today;@28.388,-25.445,16.000z
  - AFIS South African Fire Data – Regional fire data: https://www.afis.co.za/data/
  - Research Fire Dataset (arXiv reference) – Dataset and methodology reference: https://arxiv.org/abs/2412.11555

- **Infrastructure & Human Settlements**
  - OpenStreetMap (OSM) – Roads, buildings, rivers, power lines: https://www.openstreetmap.org/

- **Weather & Fire Weather Index**
  - ERA5 Reanalysis (ECMWF) – Global historical and real-time weather: wind, temperature, humidity, precipitation: https://cds.climate.copernicus.eu/cdsapp#!/dataset/reanalysis-era5-single-levels
  - South African Weather Service (SAWS) – Local weather feeds for real-time predictions: https://www.weathersa.co.za/

- **Vegetation & Fuel Load**
  - MODIS Vegetation Indices (NDVI/EVI) – Vegetation greenness for fuel condition: https://earthdata.nasa.gov/
  - SMAP Soil Moisture Dataset (NASA) – Soil moisture to estimate dryness: https://nsidc.org/data/smap
  - Land Cover Data (ESA WorldCover / Copernicus) – Fuel type classification: https://esa-worldcover.org/

- **Terrain / Topography**
  - ASTER GDEM – Alternative elevation dataset: https://asterweb.jpl.nasa.gov/gdem.asp

2.4.2 Real-Time Processing

Fire spread predictions must be generated quickly enough to remain useful during an active incident, placing a hard limit on the processing time available for terrain and spread calculations.

2.4.3 Third-Party Dependency

The system depends on an external weather data provider that it does not control, hence it is exposed to that provider's rate limits, latency, and potential downtime.

2.4.4 Data Currency

When the external weather source is unavailable, the system must continue operating using previously cached data, which means predictions may temporarily rely on information that is no longer current.

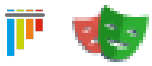
2.4.5 Shared Data Access

Multiple components that read and write fire-prediction data need access to a shared, up-to-date data source, which constrains how independently those components can scale or operate without risk of data conflicts or contention.

2.5 Mapping Quality Requirements to Architectural Decisions

Quality Requirement	**Architectural Decision**
Scalability (Support 1000+ concurrent users)	Horizontal scaling mechanisms within the distributed system layers.
Security (AES-256, RBAC, API auth)	Security Proxy for traffic validation and authentication controller.
Availability (99.9% uptime, local caching)	Layered architecture with local client-side caching and resillient data synchronization.
Performance (3s prediction, 95% of simulation requests rendered)	Blackboard architecture enabling collaborative, asynchronous data processing.
Reliability (Recover within 2 mins, handle input loss)	Circuit Breaker pattern on Weather Data Adapter to manage external service failures.
Maintainability (Integrate new data/fire risk models in <12 hours)	Adapter pattern and modular Service Layer design isolating external dependencies.

3 Technology Requirements

Layer	Technology	
Frontend	React, Next.js, DaisyUI	
Backend	Python, FastAPI	
Mapping	Mapbox GL	
AI/ML	PyTorch	
Database	PostgreSQL, PostGIS, SQLAlchemy	
Cache	Valkey	
Notifications	AWS SNS	
Versioning	Git, GitFlow	
Containerization	Docker	
Deployment	Amazon Web Services	
Testing	Pytest, Playwright	

3.1 Technology Requirements Choices

3.1.1 Frontend

- React
- Next.js
- DaisyUI

React's concurrent rendering handles simultaneous live map updates without bottlenecks. Next.js SSR ensures WCAG 2.1 compliance as screen readers require fully rendered HTML. DaisyUI handles the structural baseline and the state logic of UI elements, allowing to treat the frontend as a modular, plug-and-play system.

3.1.2 Backend

- Python
- FastAPI

Python has mature support for the full AI/ML stack required (PyTorch, scikit-learn, GeoPandas, GDAL). FastAPI's async model concurrently ingests live ERA5, SAWS, and NASA FIRMS data without blocking, and runs at 21,000+ requests/second (above the 1,000 concurrent user requirement). Pydantic validation meets the 200ms form validation delivery requirement at the API layer.

3.1.3 Mapping

- Mapbox GL

GPU-accelerated WebGL rendering redraws fire spread polygons and overlays in under 16ms. Built-in offline tile packs allow firefighters to use the map without internet. The library is open source and free, with a 50000 map loads/month free tier.

3.1.4 AI/ML

- PyTorch

PyTorch's CNN achieves F1 scores of 0.64 - 0.72 on fire spread benchmarks. ts LSTM layer handles the 30-day vegetation dryness and multi-hour wind forecasts required by the 24 - 72hr risk engine. The autograd engine enables post-event model fine-tuning.

3.1.5 Database

- PostgreSQL
- PostGIS
- SQLAlchemy

PostGIS provides 1 000+ geospatial functions for cross-referencing fire paths with OSM roads and settlements. ACID transactions guarantee complete audit records with rollback capability, ensuring firefighter action logging. GiST spatial indexes enable sub-second proximity queries powering automated infrastructure alerts. SQLAlchemy provides a secure, programmatic database abstraction layer that natively integrates with spatial extensions to handle complex geospatial coordinate data, like fire pins and containment lines, using clean Python code. Additionally, its built-in connection pooling and automated parameter sanitization safeguard your platform against SQL injection while maintaining high performance during sudden concurrent traffic spikes.

3.1.6 Cache

- Valkey

Valkey operates entirely in-memory, delivering sub-millisecond read times. When the AI simulation engine needs wind, humidity, and vegetation data instantly, pulling it from Valkey rather than making repeated external API calls or querying a disk-based SQL database ensures the prediction grid is generated within seconds.

3.1.7 Notifications

- AWS SNS

SNS includes robust, configurable delivery retry policies out of the box. If FCM fails to accept a message, SNS automatically retains the payload and executes an exponential backoff strategy to ensure the alert is eventually delivered, eliminating the need to write custom retry logic in our API.

3.1.8 Versioning

- Git
- GitFlow

Git paired with Gitflow provides isolated, risk-free environments to develop complex features like the AI simulation engine without compromising the stability of the live, safety-critical application. It establishes a dedicated pipeline for rigorous testing and rapid emergency hotfixes, ensuring the platform remains highly reliable and continuously available for field operations.

3.1.9 Containerization

- Docker

Docker containerization locks down all complex geospatial and AI dependencies into an identical environment across development and production, eliminating configuration errors and safeguarding system reliability. It enables lightweight, isolated services that can spin up in milliseconds to horizontally scale under sudden user load during active fire emergencies without risking core system downtime.

3.1.10 Deployment

- Amazon Web Services

The AWS Free Tier offers a zero-cost cloud environment to deploy and integrate our multi-service architecture, allowing us to test critical components like web hosting, file storage and push notification brokers without any upfront investment. It also provides immediate access to enterprise-grade security tools, including free SSL/TLS certificates and identity access management, letting us fully validate our system's functional capabilities before scaling up to paid infrastructure

3.1.11 Testing

- PyTest
- Playwright

Pytest allows us to rigorously validate the backend API, server-side role checks, and complex simulation edge-cases using reusable fixtures and parameterized data inputs. Meanwhile, Playwright drives actual browsers to emulate real-world field conditions, ensuring responsive map layouts, touch-drawn containment lines, and offline-caching layers work flawlessly for emergency responders.

4 API Contracts

4.1 Authentication & Access Control Service

Description: Manages user registration, login, and secure session management using JWT tokens.

Inputs:

- **email** (string) − User's unique email address.
- **password** (string) − User's secure password.

Outputs:

- **access_token** (JWT) – Token used for all subsequent authenticated API calls.
- **role** (string) – The assigned user role (e.g., Registered User, Firefighter, Admin).

Usage / Interaction Rules:

- Accessed via HTTPS/TLS.
- Tokens are invalidated immediately upon logout or role change.
- Authentication events are written to an audit log.

4.2 Fire Reporting & Verification Service

Description: Processes new fire reports from users and manages the verification status of incidents.

Inputs:

- **location** (coordinates) – GPS pin location of the fire.
- **timestamp** (DateTime) – Time the report is submitted.
- **fire_size** (string/int) – Estimated size of the fire.
- **photo** (file, optional) – Image attachment of the fire.
- **description** (string, optional) – Free-text details provided by the reporter.

Outputs:

- **reference_number** (string) – TA unique identifier for the report.
- **is_duplicate** (boolean) – Returns true if a report is a duplicate.
- **status** (string) – Initial status set to "Pending".

Usage / Interaction Rules:

- Can be called by anonymous (unregistered) users.
- Duplicate reports at the same location are automatically linked to one incident.

4.3 AI Fire Spread Simulation Engine

Description: Calculates predicted fire spread horizons based on environmental data and terrain.

Inputs:

- **fire_location** (coordinates) – The verified origin of the fire.
- **weather_data** (object) – Includes wind speed, direction, temperature, and humidity.
- **terrain_data** (object) – Includes slope and vegetation dryness index.

- **barriers** (array, optional) − Coordinates of logged containment lines.

Outputs:

- **spread_grid** (JSON/GeoJSON) − Colourcoded probability grid for 1h, 3h, 6h, and 24h horizons.

Usage / Interaction Rules:

- Triggered automatically once a fire status changes to "Verified".
- Re-runs automatically when new weather data or containment lines are received.

4.4 Live Fire Map Service

Description: Provides real-time data for fire markers and spread overlays to the client application.

Inputs:

- **user_role** (string) − Used to determine if the requester can see the spread overlay.
- **map_bounds** (coordinates) − The current viewport to filter visible markers.

Outputs:

- **active_fires** (list) − Array of verified fire markers with status, time, and size.
- **overlay_data** (object) − Prediction grid (visible to Registered Users only).

Usage / Interaction Rules:

- Service must support auto-refresh to push new data without manual reloads.
- Pending fires are filtered out and not shown to the public.

4.5 Proximity Alert Service

Description: Dispatches push notifications to users within a specific risk zone of a verified fire.

Inputs:

- **fire_id** (string) − Reference to the verified fire incident.
- **zone_id** (string) − The grid zone identified as at-risk.

Outputs:

- **delivery_status** (boolean) − Confirmation that the message was sent to the broker.

Usage / Interaction Rules:

- Must dispatch notifications via Firebase Cloud Messaging (FCM) within 5 seconds of verification.
- Messages must include a deep-link to the specific fire on the map.

4.6 Firefighter Tactical Service

Description: Handles specialized inputs from field personnel, such as GPS tracking and containment logging.

Inputs:

- **firefighter_id** (string) – Unique ID of the responder.
- **current_gps** (coordinates) – Real-time location of the firefighter.
- **containment_line** (array) – Set of GPS coordinates representing a firebreak.

Outputs:

- **sync_status** (string) – Confirmation of data storage and simulation trigger status.

Usage / Interaction Rules:

- Strictly restricted to users with the "Firefighter" role.
- Every logged action must be stored with a high-precision timestamp for auditing.

5 Deployment

5.1 Deployment Requirements

5.1.1 Live, Accessible System

Fire Away is accessible at: https://fireaway.csml.co.za

5.1.2 Environment Parity

- **Local Development:** All new features are first developed in our local environment in a Docker container.
- **Staging:** Code is pulled into the staging environment once all CI tests pass in local development. This environment is to remain stable for production.
- **Production:** This environment is stable and is the public release of the app.

5.1.3 Rollback Strategy

Our system infrastructure uses a Blue-Green Deployment rollback strategy that is backed by Caddy as a dynamic reverse proxy. Traffic is directed to one of two identical environments (slots blue or green) while the other remains idle.

Rollback Procedure

1. Re-initialize the previous slot: Start the stopped containers on the target rollback slot.
2. Network Re-attachment: Reconnect Caddy to the previous slot's network.
3. Proxy Swap: Swap Caddy's configuration back to the previous upstream target and issue a reload.
4. Isolate Failure: Stop the failing current slot for debugging.

5.2 Deployment Diagram

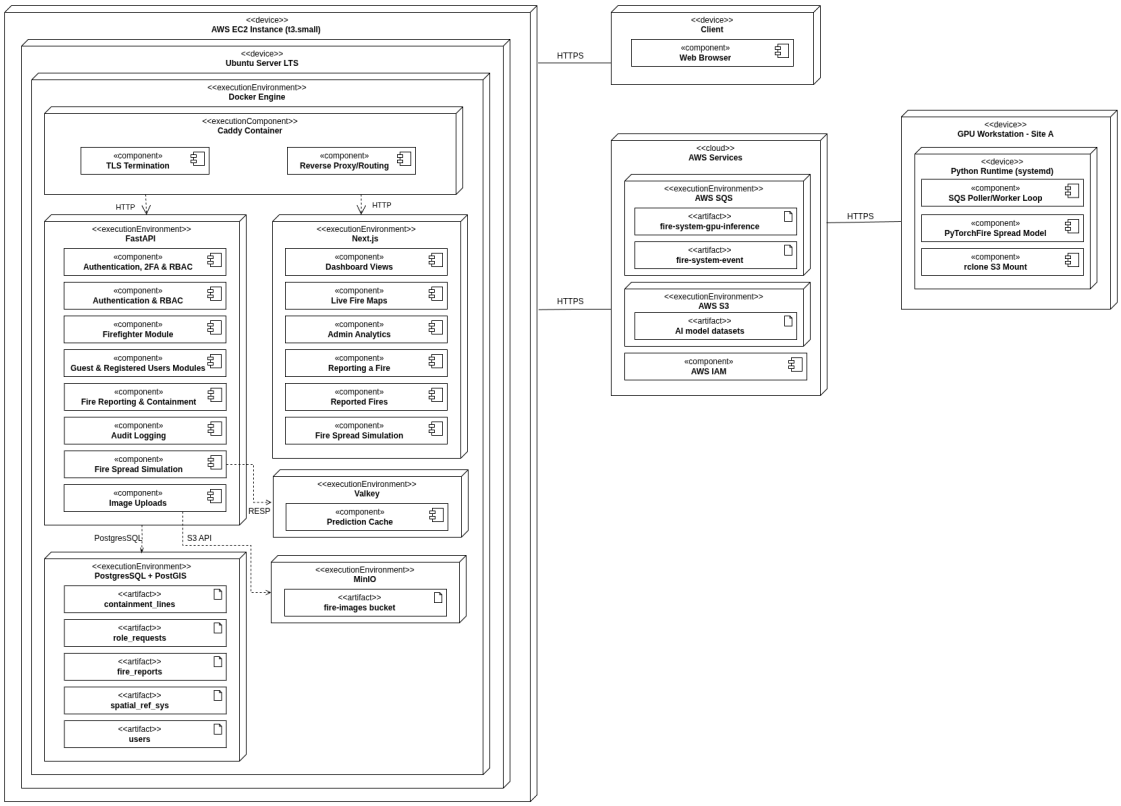


Figure 2: Deployment Diagram

5.3 CI/CD Pipeline Diagram

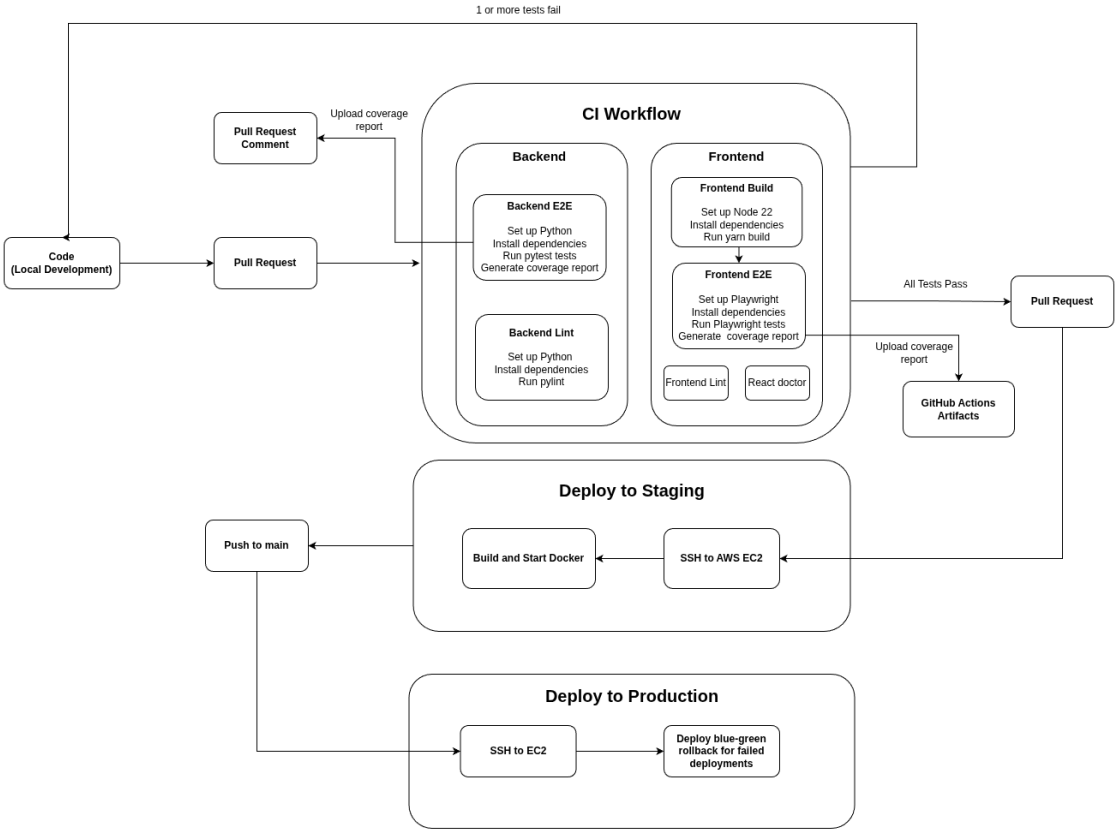


Figure 3: CI/CD Pipeline Diagram

6 Change Log

Date	Version	Change
July	1.0	Initial version
August	2.0	Architectural Requirements